

RAG-Dokumentation

Index, Suche, Kette und Sprachfassungen

Zweck

Die RAG-Schicht liegt in `rag_project.app`: `rag_lib.py` trägt die Bibliothek, `rag_app.py` die Bedienung, `rag_utils.py` die Formatierer für lange Texte. Sie baut auf den drei Vorverarbeitungsstufen des Projekts auf und ergänzt genau das, was das Projekt selbst nicht hat — den Weg von den Chunks in einen Vektorindex und die Kette darüber.

Der Pfad ist Teil der Änderung: Alle drei Module liegen heute unter `app/`. Verweise auf ein flaches `rag_project.rag_lib` sind veraltet.

Die fünf Stufen

Stufe	Modul	Baustein
laden	<code>pipeline/loading.py</code>	<code>load_documents</code>
bereinigen	<code>pipeline/cleaning.py</code>	<code>clean_documents</code>
chunken	<code>pipeline/chunking.py</code>	<code>split_documents</code>
indexieren	<code>app/rag_lib.py</code>	<code>build_vectorstore</code>
antworten	<code>app/rag_lib.py</code>	<code>build_rag_chain</code>

Die ersten drei Stufen stammen aus dem Projekt und sind hier nicht neu implementiert. Nur die letzten beiden liegen in der RAG-Schicht. Diese Trennung ist der Grund, warum sich die Vorverarbeitung auch ohne Netz und ohne Schlüssel prüfen lässt: Sie ist ein eigener, vollständig lokaler Teil.

Der Index

Der Index wird bei jedem Lauf neu aufgebaut und lebt nur im Arbeitsspeicher — die Adresse lautet `:memory:`. Es gibt keine Persistenz und keinen geteilten Zustand zwischen zwei Läufen. Das ist für die Prüfung von Chunking und Suche bequem, weil kein alter Index ein Ergebnis verfälschen kann; für einen dauerhaften Betrieb wäre es nicht geeignet.

Der Sammlungsname entsteht aus der Sprache und dem Suchmodus. Damit kollidieren zwei Läufe im selben Prozess nicht, wenn beide Modi gebaut werden — was beim Vergleich aller Fragen tatsächlich passiert.

Suchmodi

Modus	Verfahren	Kenntnis
dense	Ähnlichkeitssuche über die Einbettungen	erkennt Umschreibungen und Synonyme
hybrid	Einbettungen plus Stichwortsuche, beide Anteile werden zusammengeführt	findet zusätzlich seltene Wörter und Eigennamen

Für die hybride Suche kommt ein Stichwortmodell hinzu. Ohne es fehlt die zweite Komponente, und der Modus ist nicht sinnvoll zu bauen — der Store lehnt ihn dann ab. Der Unterschied zwischen beiden Modi ist der eigentliche Gegenstand des Vergleichs in --compare.

Einbettungsmodelle

Die Indexierung nutzt ein eigenes Modell, das nicht mit dem antwortenden Modell identisch sein muss. Zur Wahl stehen zwei Modelle; beide werden über dieselbe Adresse angesprochen. Ein Katalogeintrag, der nur Text erzeugt und keine Einbettung liefert, ist ausdrücklich kein Einbettungsmodell — der Aufruf wird mit Begründung abgewiesen, statt einen unbrauchbaren Index zu bauen.

Rolle	Wahl
Einbettung (Standard)	openai/text-embedding-3-small
Einbettung (Alternative)	nvidia/llama-nemotron-embed-vl-1b-v2
Stichwortanteil der hybriden Suche	Qdrant/bm25
Antwortmodell	ibm-granite/granite-4.2-8b ohne explizites Nachdenken

Einbettung ist nicht Chat

- Ein Chat-Modell liefert keine Vektoren. Wird eines als Einbettungsmodell angefragt, meldet die Anwendung den Grund und nennt die verfügbaren Modelle.

Der Kontextblock

Bevor die Treffer an das Modell gehen, werden sie zu einem Kontextblock zusammengefügt. Jeder Treffer bekommt eine Herkunftszeile mit Kapitel, Abschnitt und Titel. Damit ist im Prompt sichtbar, woher ein Treffer stammt — auch dann, wenn die Überschrift im Text selbst keine Nummer mehr trägt, wie es in der englischen Fassung der Fall ist.

Die Kette

Die Kette schiebt die Frage an zwei Stellen weiter: einmal durch den Retriever, einmal unverändert in den Prompt. Der Prompt besteht aus einer System-Nachricht mit dem Kontextplatzhalter und einer Nutzer-Nachricht mit der Frage. Am Ende steht ein Parser, der die Antwort auf reinen Text reduziert.

Der System-Prompt ist bewusst streng: Er verlangt, sich auf den Kontext zu stützen, und gibt einen festen Satz vor, wenn der Kontext die Antwort nicht enthält. Antworten, die diesen Satz tragen, zählt die Anwendung als Grenze des Korpus benannt.

Sprachfassungen

Deutsch und Englisch liegen als getrennte Wissensbasen vor — je ein Quelldokument in zwei Formaten und je ein eigener System-Prompt. Auch die Testfragen sind je Sprache getrennt gepflegt. Deutsch ist die Werkseinstellung.

Sprache	PDF	Text
de	docs/handbuch.pdf	docs/handbuch.txt
en	docs/handbook.pdf	docs/handbook.txt

Die englischen Kapitelüberschriften tragen keine Nummern. Deshalb bleiben chapter und section in den Chunk-Metadaten dort leer, während sie in der deutschen Fassung gefüllt sind. Ein Vergleich von Trefferzahlen zwischen beiden Sprachen muss das berücksichtigen — in der deutschen Fassung ist jedem Fund eine Stelle zugeordnet, in der englischen nicht.

Testfragen

Je Sprache sind vier Fragen gepflegt. Die ersten drei sind aus dem Handbuch beantwortbar; die letzte liegt bewusst außerhalb des Korpus und prüft, ob die Anwendung die Grenze benennt, statt eine Antwort zu erfinden.

Aufrufe

Aufruf	Wirkung
--------	---------

Aufruf	Wirkung
uv run rag	Übersicht: Ladebericht, Bilanz, Chunking, Vorschau
uv run rag --config	aktive Einstellungen ohne Klartext-Keys
uv run rag --queries	die Testfragen der Sprache zeigen
uv run rag --load	Stufe 1: Ladebericht
uv run rag --clean	Stufe 2: Reinigungsbilanz
uv run rag --chunks	Stufe 3: Chunking-Kennzahlen
uv run rag --preview	die ersten Chunks mit Herkunft
uv run rag --write DATEI.md	Chunks als Markdown ablegen
uv run rag --retrieve "Frage"	Treffer zeigen, ohne das Modell zu rufen
uv run rag --ask "Frage"	eine Frage beantworten
uv run rag --ask "Frage" --show-hits	Antwort mit Belegstellen
uv run rag --compare	alle Testfragen in beiden Suchmodi

Fortschritt und Fehler gehen nach stderr, Ergebnisse nach stdout. So bleibt die eigentliche Ausgabe weiterleitbar, während der Fortschritt sichtbar bleibt. Mit --no-progress lassen sich die Fortschrittszeilen ganz unterdrücken.

Der Vergleich

Der Vergleich beantwortet jede Testfrage in beiden Suchmodi und stellt die Ergebnisse nebeneinander: die Kapitel der Treffer und die Antwort. Am Ende steht eine Übersicht mit der durchschnittlichen Trefferzahl je Modus und der Zahl der Antworten, die die Korpusgrenze benannt haben.

Dieser Vergleich ist ausdrücklich eine Beobachtung, keine Bewertung. Es gibt keine Richter-Modelle, keine Metriken und keine Noten — nur die Gegenüberstellung dessen, was beide Modi liefern. Wer eine Bewertung braucht, muss sie aus den gezeigten Antworten selbst ableiten.

Fehlerverhalten

Fehler	Bedeutung	Exit-Code
MissingCredentialsError	Schlüssel fehlt	1

Fehler	Bedeutung	Exit-Code
IndexBuildError	Store ließ sich nicht aufbauen	2
RagError	Basisfehler, etwa unbekannte Sprache	1
DocumentPathError	Quelldokument fehlt	1
LoaderDependencyError	Loader-Paket fehlt	2

Beispielablauf

Ein sinnvoller Einstieg prüft zuerst die drei lokalen Stufen und geht erst dann ins Netz. So ist ein Fehler in den Chunks ausgeschlossen, bevor ein Modellaufruf bezahlt wird.

- `uv run rag --chunks` — Vorverarbeitung prüfen, ohne Netz
- `uv run rag --preview` — Herkunft der ersten Chunks ansehen
- `uv run rag --retrieve "FRAGE"` — Treffer prüfen, ohne das Modell zu rufen
- `uv run rag --ask "FRAGE" --show-hits` — eine Frage mit Belegstellen beantworten
- `uv run rag --compare` — beide Suchmodi über alle Testfragen gegenüberstellen